TI Garantias

Portal Interno de Garantias, Retenciones y Devolucion de TI

Proyecto final - Practicas DevOps

Grupo 1: Armando Chavez, Geovanny Robles, Guido Vallejo

Angular + .NET 8 + PostgreSQL + Docker + Terraform + Jenkins + Azure

Contexto y Objetivo

Que problema resolvemos

Situacion inicial

- Procesos de garantías, retenciones y devoluciones con trazabilidad limitada
- Necesidad de centralizar facturas, contratos, entregables.
- Riesgo operativo al depender de despliegues manuales
- Falta de visibilidad rápida sobre salud, métricas y logs

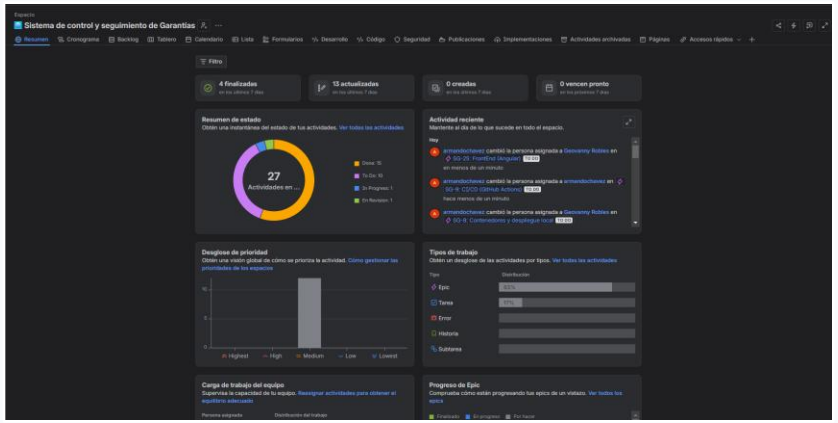
Objetivo del piloto

- Construir un portal interno seguro y auditable
- Automatizar Build, pruebas, empaquetado y despliegue
- Manejar infraestructura como Código
- Operar en Azure Container Apps con monitoreo y secretos centralizados

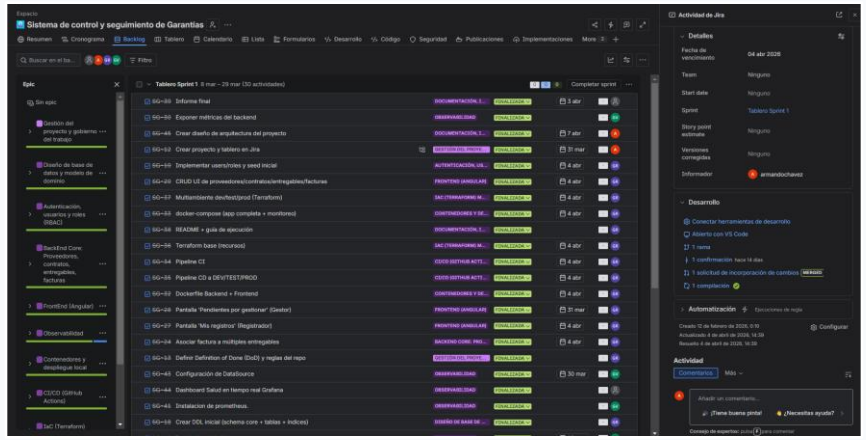
Gestion del Trabajo

Gestion del Trabajo

- Backlog gestionado en Jira con epicas, tareas y responsables.
- Seguimiento del sprint y avance del equipo.
- Priorizacion por frentes funcionales y tecnicos.
- Trazabilidad entre Jira y GitHub.
- Evidencia de ramas, commit, pull request y compilacion asociadas a tareas.



Vista general de Jira: backlog, actividad y seguimiento del espacio.

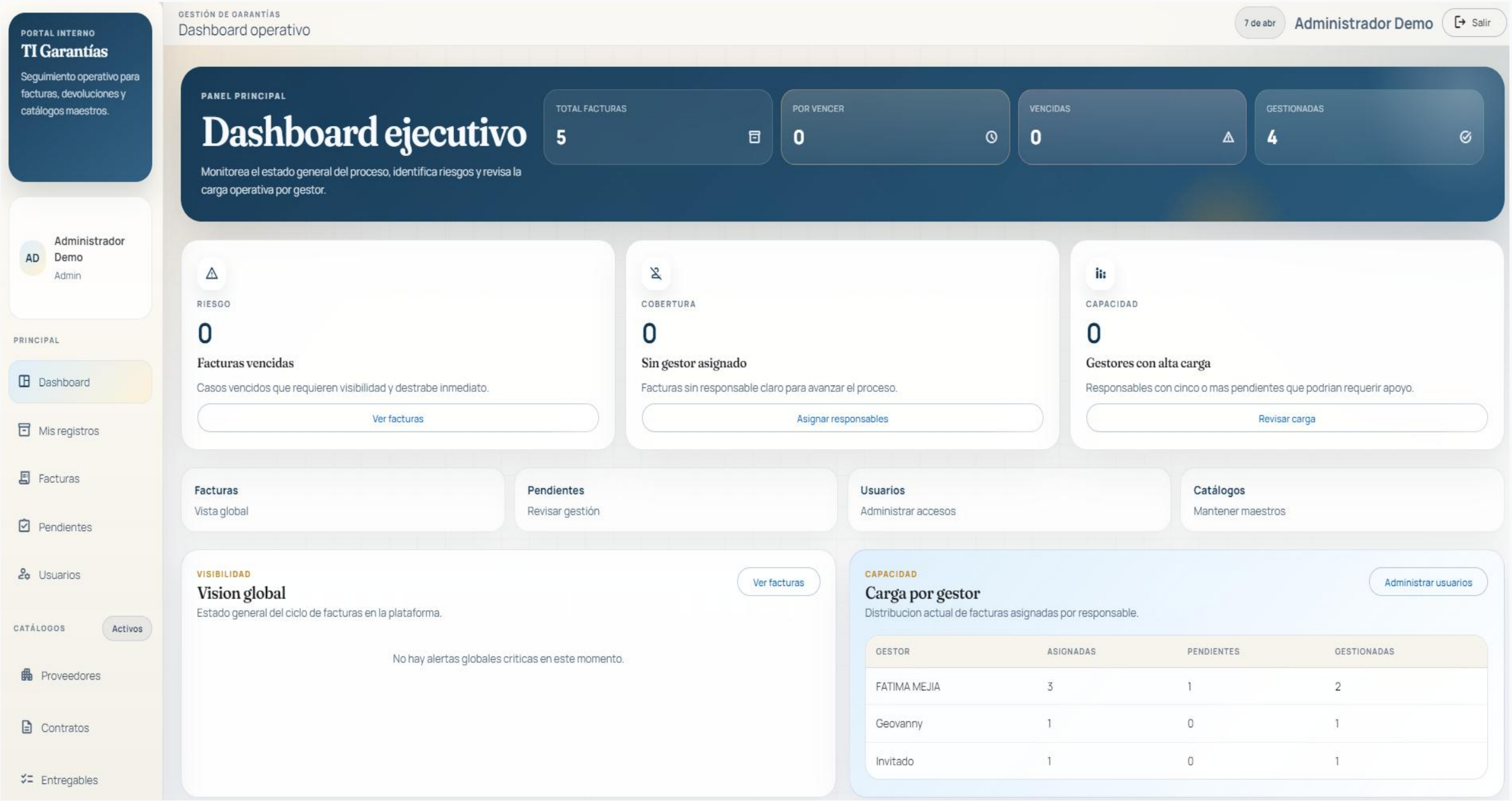


Detalle de tarea vinculada a GitHub: rama, commit, pull request y compilacion.

Estas capturas evidencian que el trabajo se gestiono en Jira con visibilidad de backlog, sprint y responsables. Ademas, ciertas tareas quedaban conectadas con GitHub, lo que permitia seguir el desarrollo y la integracion continua desde la misma herramienta.

Solucion Propuesta

Que problema resolvemos



Stack Tecnico

Componentes principales

Frontend

Angular 20
Angular Material
Nginx proxy
Variables por ambiente

Backend

ASP.NET Core .NET 8
EF Core
JWT + RBAC
Hangfire

Datos

PostgreSQL 16
Schema core
Adjuntos fuera de BD
Azure Files

DevOps

Docker Compose
Jenkins
GitHub Actions
Terraform + Azure

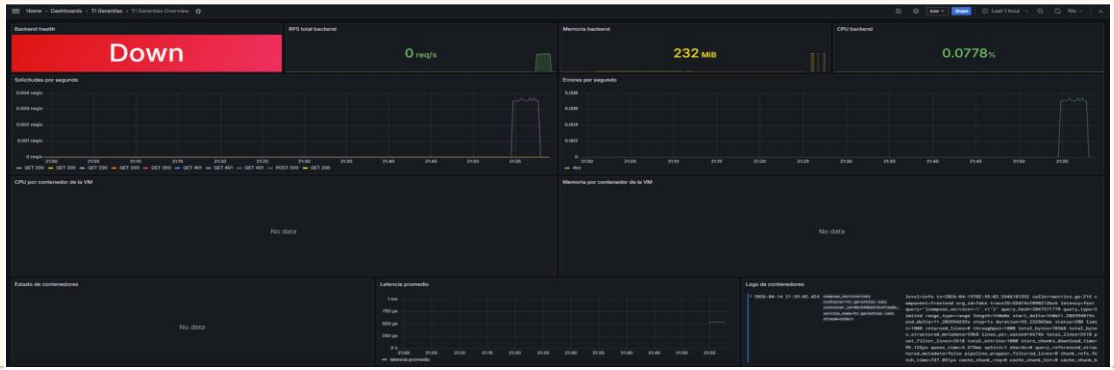
Monitoreo

Grafana
Prometheus
Loki
Promtail
cAdvisor

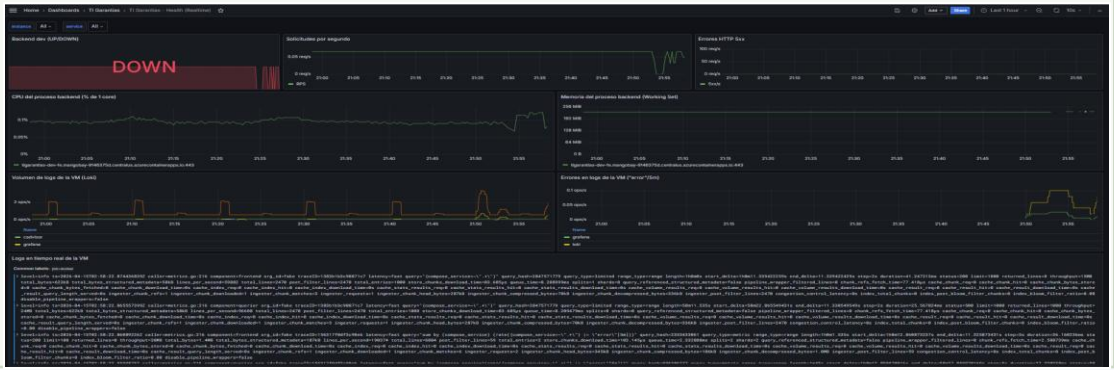
Observabilidad y Dashboards

Evidencia visual de monitoreo y fuentes de datos

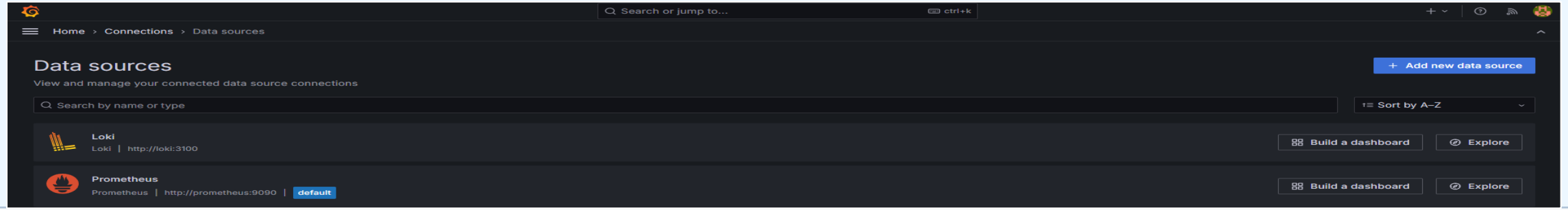
Grafana Overview



Grafana Health



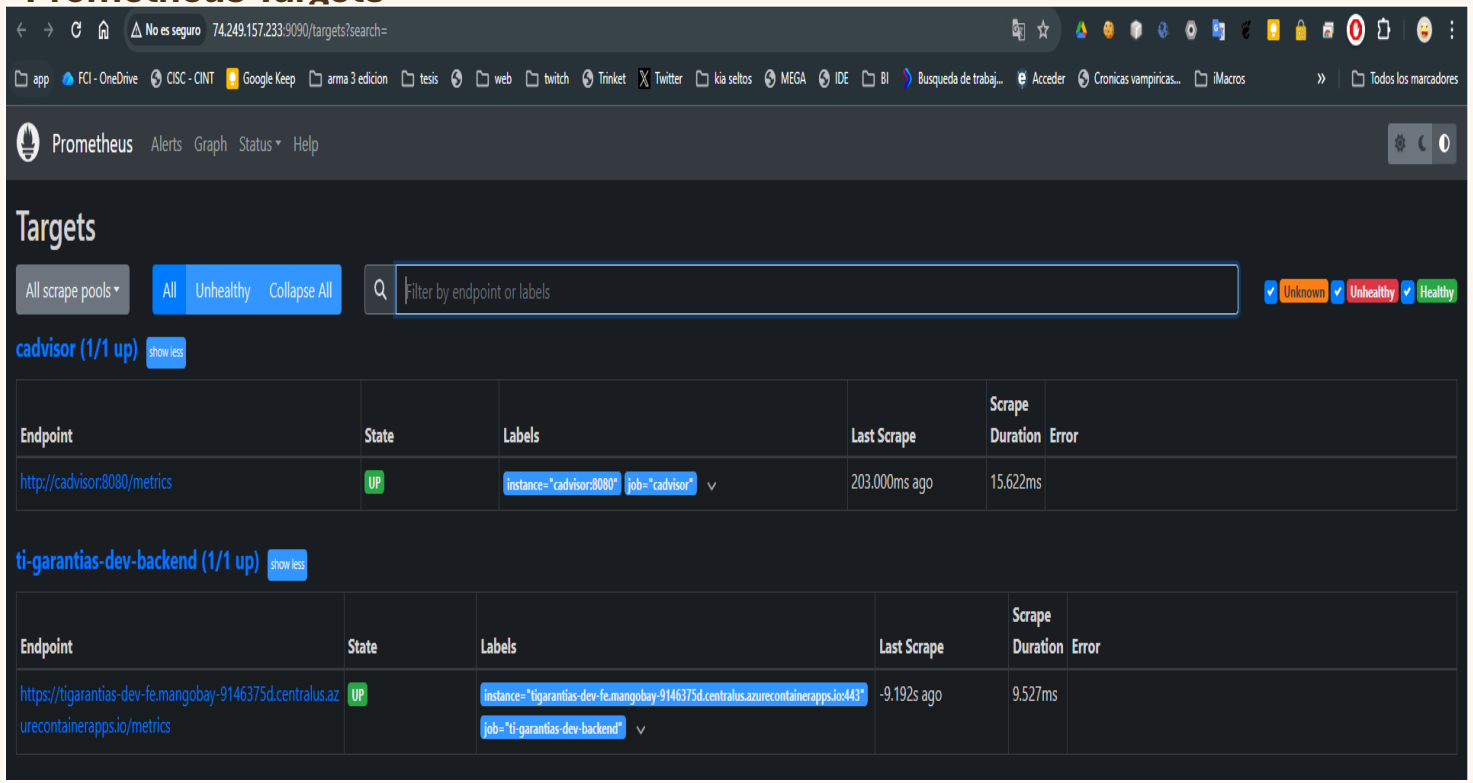
Grafana Data Sources



Estos dashboards evidencian la capa de observabilidad del proyecto. Grafana centraliza la consulta de metricas y logs usando Prometheus y Loki como fuentes de datos, permitiendo visualizar salud, errores, latencia y actividad del entorno.

Prometheus y Targets

Prometheus Targets



Que muestra esta pantalla

- Prometheus consulta endpoints llamados targets.
- En la captura se ve cAdvisor en estado UP.
- Tambien se ve el backend monitoreado como ti-garantias-dev-backend.
- Si el target responde, Prometheus puede recolectar metricas y Grafana luego las visualiza.
- Esta pantalla sirve para validar que el monitoreo esta realmente conectado.

Flujo de metricas Backend -> /metrics -> Prometheus -> Grafana | cAdvisor -> Prometheus -> Grafana

Stack de monitoreo en Docker

docker ps en la VM compartida

```
rca_pro@hotmail.com@tigarantias-jenkins-vm:~$ sudo docker ps
ONTAINER ID   IMAGE                                COMMAND                  CREATED       STATUS               PORTS                                                   NAMES
bbf0d76ca75   ti-garantias/jenkins-runner:latest   "/usr/bin/tini -- /u…"   7 days ago    Up 7 days            0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 50000/tcp   ti-garantias-jenkins
ae9e375738c   grafana/grafana:11.2.0               "/run.sh"                2 weeks ago   Up 7 days            0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp              ti-garantias-grafana
65cab82d6ab   prom/prometheus:v2.54.1              "/bin/prometheus --c…"   2 weeks ago   Up 7 days            0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp              ti-garantias-prometh
us
1b78af06edd   grafana/promtail:3.0.0               "/usr/bin/promtail -…"   2 weeks ago   Up 7 days                                                                             ti-garantias-promtai

6a75d157b09   gcr.io/cadvisor/cadvisor:v0.49.1     "/usr/bin/cadvisor -…"   2 weeks ago   Up 7 days (healthy)  0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp              ti-garantias-cadviso

bc54866576c   grafana/loki:3.0.0                   "/usr/bin/loki -conf…"   2 weeks ago   Up 7 days            0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp              ti-garantias-loki
rca_pro@hotmail.com@tigarantias-jenkins-vm:~$
```

Que evidencia esta captura

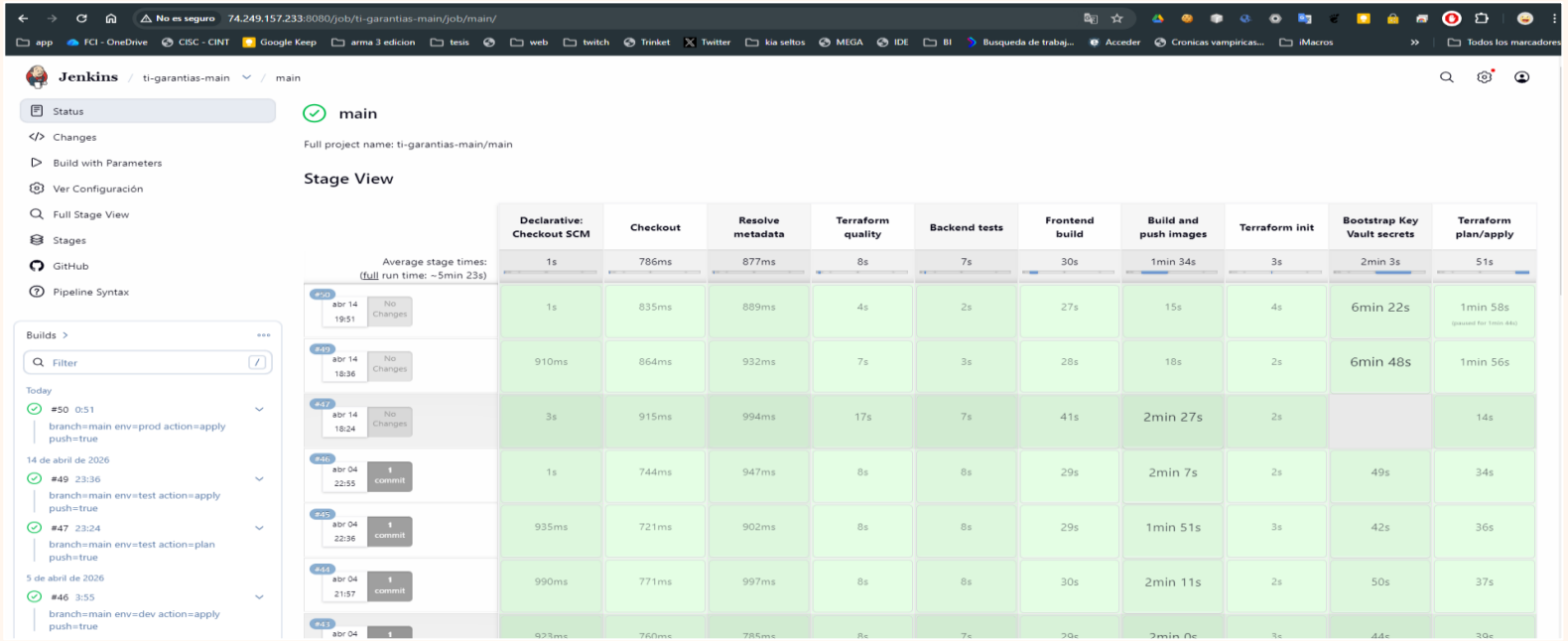
- Se observan contenedores activos de Grafana, Prometheus, Loki, Promtail y cAdvisor.
- Esto confirma que el stack corre en la VM compartida.
- Promtail envia logs a Loki y cAdvisor expone metricas a Prometheus.
- Grafana consulta Prometheus y Loki para visualizar todo.

Como se comunican

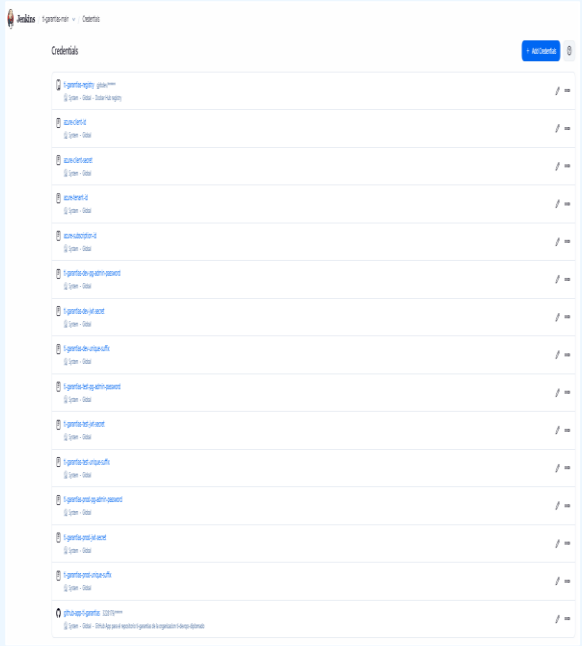
Promtail -> Loki para logs. cAdvisor -> Prometheus para metricas de contenedores. Backend -> /metrics -> Prometheus.
Grafana -> Prometheus y Loki para consultar y visualizar la informacion.

Jenkins y entrega continua

Stage View del pipeline



Credenciales de Jenkins

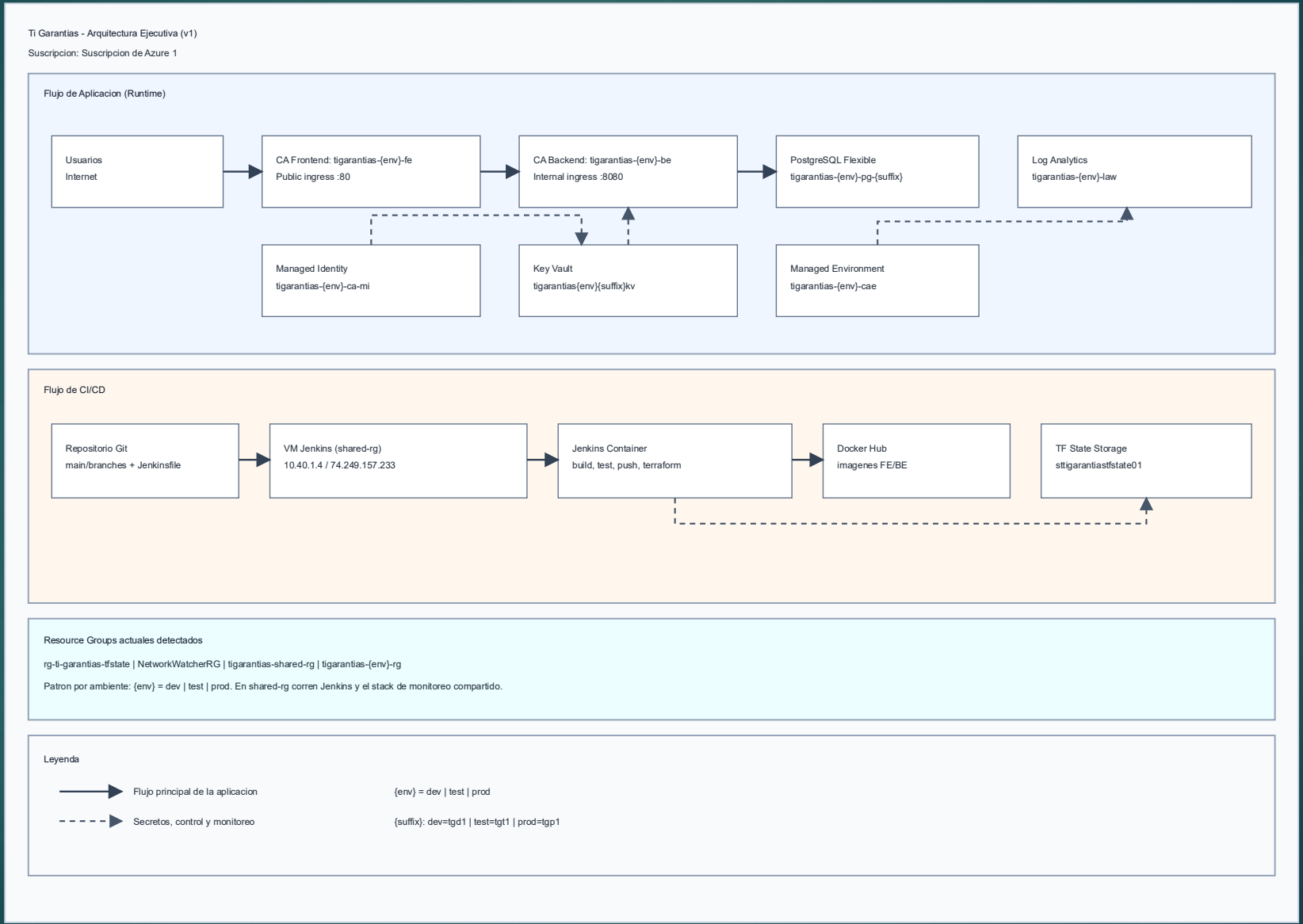


Que evidencia esta diapositiva

El Stage View muestra etapas automatizadas como checkout, validacion Terraform, pruebas backend, build frontend, push de imagenes y terraform plan/apply. La captura de credenciales evidencia que Jenkins se integraba con Azure, Docker Hub, GitHub y secretos por ambiente para ejecutar la entrega continua de forma segura.

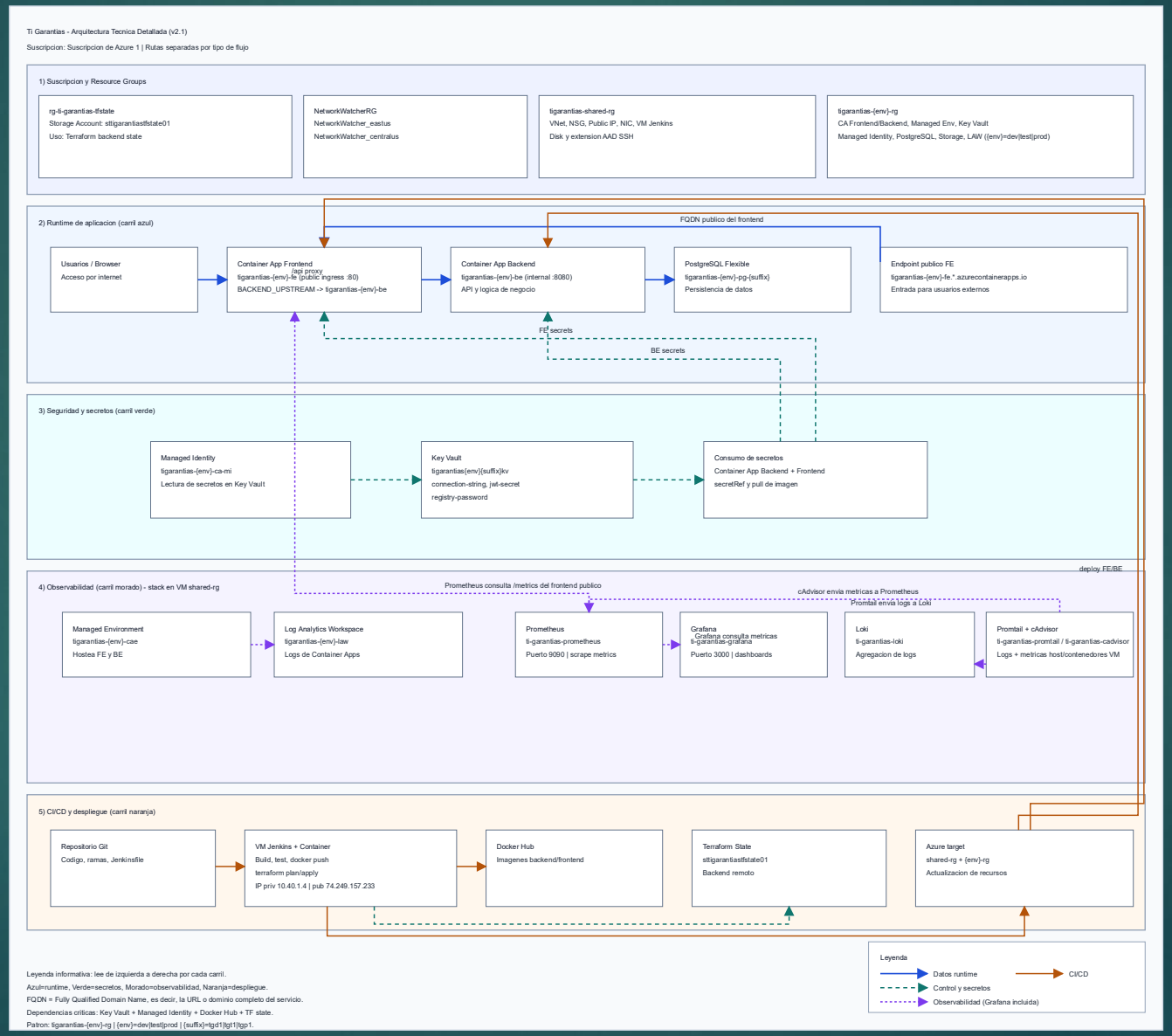
Arquitectura Ejecutiva

Vista general de la solucion



Arquitectura Tecnica

Detalle de componentes y flujos



Inventario de Recursos

Resource Groups y servicios desplegados

Ti Garantias - Inventario Visual (v3)

Suscripcion: Suscripcion de Azure 1

Suscripcion

Scope superior que contiene todos los Resource Groups listados abajo

RG: rg-ti-garantias-tfstate

Microsoft.Storage/storageAccounts
sttigarantiastfstate01
Uso: terraform state

→

Dependencias
- Jenkins pipeline consume este storage como backend remoto.
- No participa en trafico runtime de usuario.

RG: NetworkWatcherRG

NetworkWatcher_eastus
Microsoft.Network/networkWatchers

NetworkWatcher_centralus
Microsoft.Network/networkWatchers

→

Servicio administrado por Azure para diagnostico y monitoreo de red.

RG: tigarantias-shared-rg

Virtual Network (VNet)
tigarantias-shared-vnet

Network Security Group (NSG)
tigarantias-jenkins-nsg

Public IP
tigarantias-jenkins-pip

Network Interface (NIC)
tigarantias-jenkins-nic

Virtual Machine (VM)
tigarantias-jenkins-vm

→

Disk + Extension
OsDisk + AADSSHLoginForLinux

Servicios en VM (contenedores): ti-garantias-grafana, ti-garantias-prometheus, ti-garantias-loki
Tambien: ti-garantias-promtail, ti-garantias-cadvisor (logs y metricas del host/contenedores)
Puertos NSG: 22, 8080, 3000, 9090 | IP privada 10.40.1.4 | IP publica 74.249.157.233
Pipeline Jenkins consume Docker Hub y Terraform para actualizar recursos por ambiente.

RG de aplicacion: tigarantias-{env}-rg

Managed Identity
tigarantias-{env}-ca-mi

Key Vault
tigarantias{env}{suffix}kv

Storage
tigarantias{env}{suffix}st

Log Analytics
tigarantias-{env}-law

Managed Environment
tigarantias-{env}-cae

Container App FE
tigarantias-{env}-fe

→

Container App BE
tigarantias-{env}-be

PostgreSQL Flexible Server: tigarantias-{env}-pg-{suffix}
Cadena principal: FE -> BE -> PostgreSQL

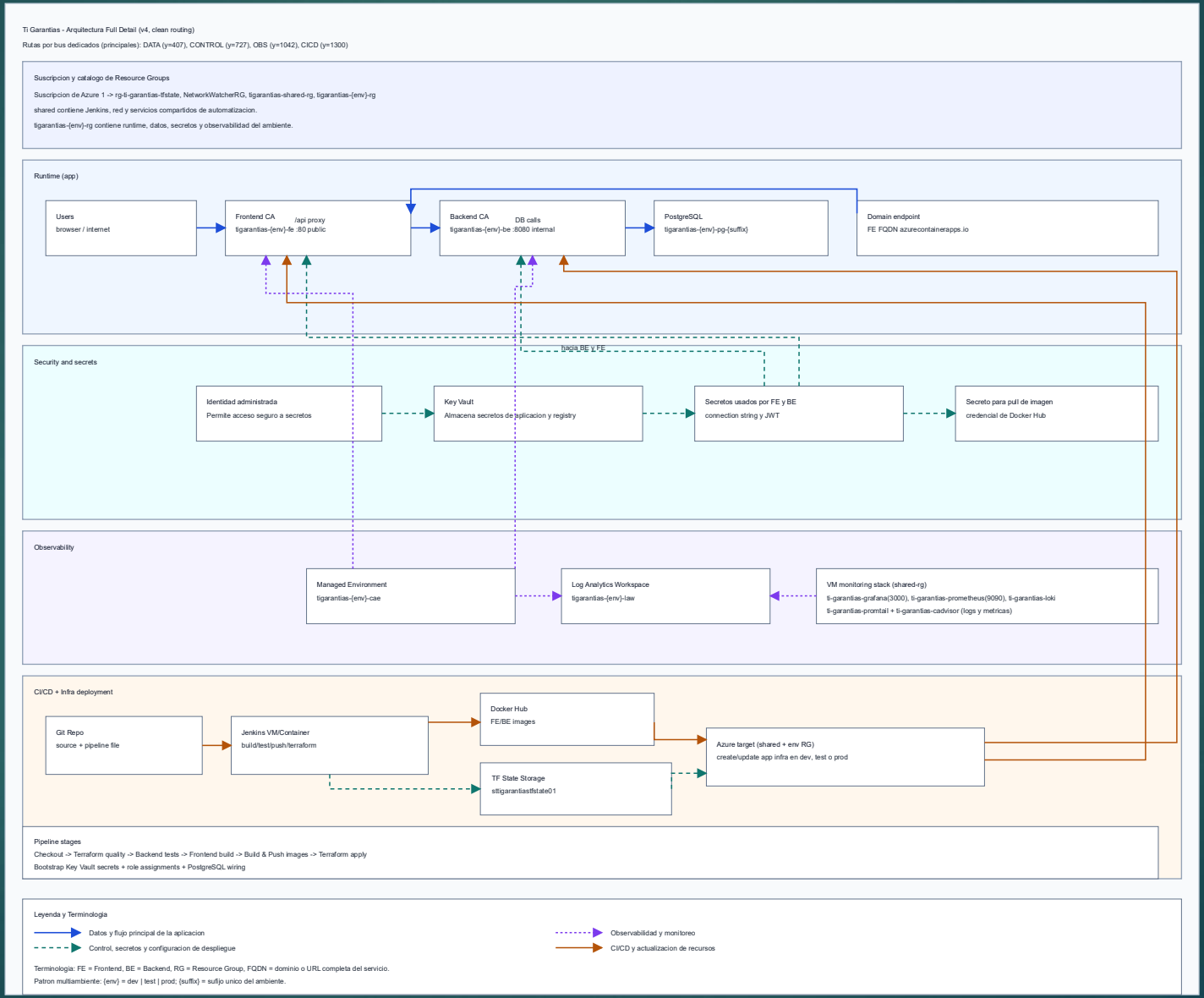
Secretos: MI -> Key Vault -> FE/BE
Logs: Managed Environment -> Log Analytics | Docker Hub suministra imagenes para Container Apps

Leyenda Informativa

Cada bloque representa un Resource Group y sus recursos listados.
Flechas: relaciones principales detectadas entre recursos.
Patron multiambiente: {env} = dev | test | prod; {suffix} = sufijo unico por ambiente.
Uso recomendado: inventario tecnico y validacion de alcance por RG.

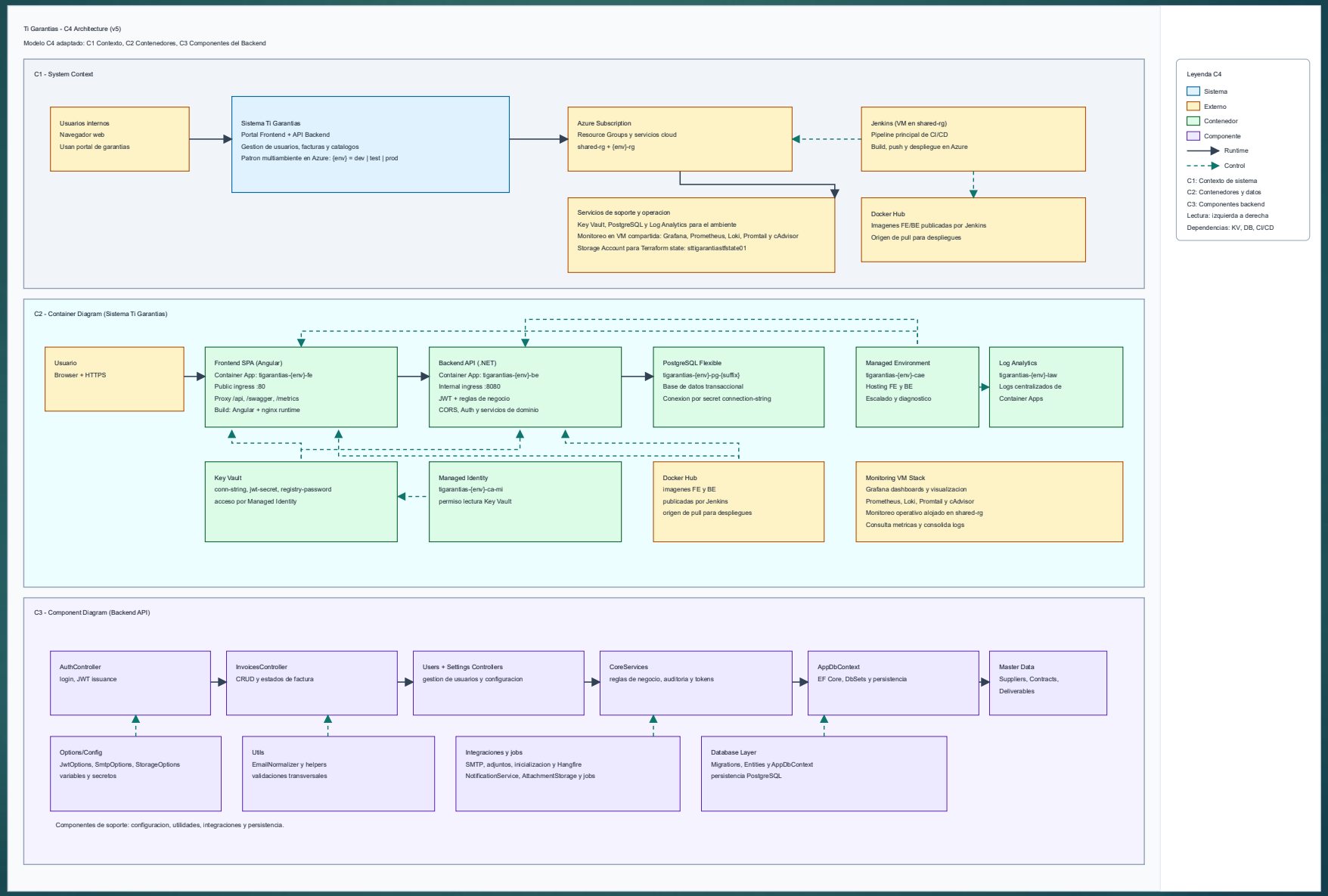
Arquitectura Full Detail

Operacion, seguridad y observabilidad



Arquitectura C4

Contexto, contenedores y componentes



Pipeline CI/CD

Validacion continua + despliegue continuo

CI

- Checkout y validacion
- Build y tests backend
- Build frontend
- Terraform fmt y validate

Flujo resumido

Commit → Actions → Jenkins → Docker → Azure

CI valida cambios; CD construye y despliega.

CD

- Versionado inmutable
- Build y push de imagenes
- Key Vault y secretos
- Terraform plan y apply

Incidencias Reales y Aprendizajes

Lo que se ajusto durante el despliegue

- Capacidad y cuota: Standard_B2s no disponible inicialmente en East US; se homologaron ambientes a centralus
- Arquitectura de VM: se descarto SKU Arm64 por incompatibilidad con el escenario
- PostgreSQL: restriccion regional LocationIsOfferRestricted en la suscripcion
- RBAC: Jenkins necesitaba permisos para roleAssignments/write sobre Key Vault
- Providers: registro de Microsoft.App y Microsoft.OperationalInsights
- Pipeline: ajuste de ansicolor, secretos, Terraform quality y verbosidad de dotnet publish
- Frontend: correccion del proxy para reenviar Host hacia el backend interno
- Register Imágenes Docker Hub Personal

Resultados Obtenidos

Valor entregado

IaC

Infraestructura versionada, reproducible y multiambiente

CI/CD

Build, pruebas, imagenes y despliegue automatizado

Ops

Monitoreo, salud, logs y secretos centralizados

- Ambiente dev desplegado y operativo en Azure Container Apps
- Aplicacion empaquetada en contenedores backend/frontend con tags inmutables
- Backend no expuesto publicamente: acceso por proxy desde frontend
- Adjuntos fuera de la base de datos y persistentes
- Documentacion tecnica, diagramas de arquitectura e informe final disponibles

Recomendaciones y Proximos Pasos

Tecnico

- Completar promoción controlada a test y producción
- Formalizar backups y restore de PostgreSQL
- Agregar escaneo de imágenes y dependencias
- Definir alertas SLO para disponibilidad y errores
-Endurecer red, políticas y secretos para producción
- Implementar validación con Sonarqube y Veracode

Gestion

- Completar datos reales de sprints y metricas
- Documentar runbooks de incidentes y despliegue
- Preparar evidencia para auditoria
- Planificar mejoras de UX segun retroalimentacion

Cierre

El proyecto demuestra una cadena DevOps integral: backlog priorizado, aplicacion funcional, infraestructura como codigo, CI/CD multiambiente y observabilidad operativa en Azure.

Gracias